Course: Operating Systems and You: Becoming a Power User
Area: Module 2 — Operating Systems
Operating System: Ubuntu Linux
Environment: UTM Virtual Machine
Author: Florencia Horita

1. Introduction

The objective of this lab is to apply fundamental concepts of user, group, and permission management in Linux using the Bash terminal.

A practical scenario was created to simulate the management of shared resources within an organization. Local user accounts and a workgroup were configured, and access permissions were assigned to files and directories.

Throughout the lab, the following tasks were performed: creating local users and groups, managing group memberships, changing ownership using `chown` and `chgrp`, and configuring permissions using `chmod` with both numeric and symbolic notation.

Access tests were also performed using different user identities to verify that the configured permissions worked as expected.

The main objective was to understand how Linux uses users, groups, and file permissions to control access to system resources.

2. Lab Environment

The lab was completed using the following environment:

- Virtualized Ubuntu Linux.
- Bash terminal.
- macOS host.
- UTM virtual machine.

Administrative operations were performed from the terminal using `sudo`, with the privileges required to create users, manage groups, and modify file and directory permissions.

3. Lab Scenario

Three local users were created to simulate a workplace environment:

- **Ana (`ana`):** member of the project team.

- **Carlos (`carlos`):** member of the project team.
- **Pedro (`pedro`):** employee outside the project team.

A local group named `projectteam` was also created to organize project team members.

The user and group assignments were configured as follows:

```
ana     → projectteam
carlos  → projectteam
pedro   → Not a member of projectteam
```

The following directory structure was then created:

```
/companylab
└── projects
    └── reporte.txt
```

The `projects` directory was configured to allow access to members of `projectteam` while restricting access for other users.

The `reporte.txt` file was used to practice ownership changes and permission modifications.

Using groups makes it possible to manage access to shared resources without assigning permissions individually to each user.

4. Creating Local Users

Three local user accounts were created to represent CompanyLab employees: `ana`, `carlos`, and `pedro`.

The `adduser` command was used with administrative privileges.

Commands used

```
sudo adduser ana
sudo adduser carlos
sudo adduser pedro
```

During the process, passwords were configured for each account.

The accounts were then verified using:

```
id ana
id carlos
id pedro
```

The output confirmed that all three accounts existed and that each user had an assigned user identifier (UID) and primary group identifier (GID).

Concepts applied

- Local user creation.

- UID and GID identifiers.
- Primary groups.
- User verification using `id`.

Figure 1 — Commands Used and Verification of Local Users

```
sudo adduser ana
sudo adduser carlos
sudo adduser pedro
flora@flora-QEMU-Virtual-Machine:~/Escritorio$ id ana
uid=1001(ana) gid=1001(ana) grupos=1001(ana),100(users),1004(projectteam)
flora@flora-QEMU-Virtual-Machine:~/Escritorio$ id carlos
uid=1002(carlos) gid=1002(carlos) grupos=1002(carlos),100(users),1004(projectteam)
flora@flora-QEMU-Virtual-Machine:~/Escritorio$ id pedro
uid=1003(pedro) gid=1003(pedro) grupos=1003(pedro),100(users)
flora@flora-QEMU-Virtual-Machine:~/Escritorio$
```

5. Creating Groups and Managing Memberships

To organize users according to their roles within CompanyLab, a local group named `projectteam` was created for project team members.

Command used

```
sudo groupadd projectteam
```

Ana and Carlos were then added to the group using `usermod`.

```
sudo usermod -aG projectteam ana
sudo usermod -aG projectteam carlos
```

The `-aG` options add a user to a supplementary group without removing existing group memberships.

The configuration was verified using:

```
id ana
id carlos
id pedro
```

The results confirmed that Ana and Carlos belonged to `projectteam`, while Pedro was not a member of the group.

Concepts applied

- Local group creation.
- Group membership management using `usermod`.

- Primary and supplementary groups.
- Membership verification using `id`.

Figure 2 — Commands Used and Verification of projectteam Memberships

```
sudo groupadd projectteam
sudo usermod -aG projectteam ana
sudo usermod -aG projectteam carlos
flora@flora-QEMU-Virtual-Machine:~/Escritorio$ id ana
uid=1001(ana) gid=1001(ana) grupos=1001(ana),100(users),1004(projectteam)
flora@flora-QEMU-Virtual-Machine:~/Escritorio$ id carlos
uid=1002(carlos) gid=1002(carlos) grupos=1002(carlos),100(users),1004(projectteam)
flora@flora-QEMU-Virtual-Machine:~/Escritorio$ id pedro
uid=1003(pedro) gid=1003(pedro) grupos=1003(pedro),100(users)
flora@flora-QEMU-Virtual-Machine:~/Escritorio$
```

6. Creating the Directory Structure

A directory structure was created to organize CompanyLab's shared project resources.

The following structure was used:

```
/companylab
└── projects
```

The directories were created using `mkdir` with absolute paths.

Commands used

```
sudo mkdir /companylab
sudo mkdir /companylab/projects
```

The structure was then verified using:

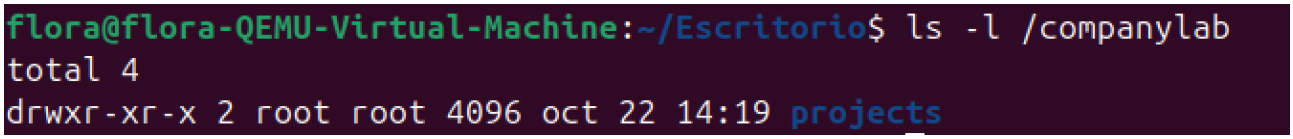
```
ls -l /
ls -l /companylab
```

The output confirmed that the `projects` directory existed inside `companylab`, initially with `root` as both its user and group owner.

Concepts applied

- Directory creation using `mkdir`.
- Hierarchical resource organization.
- Absolute paths.
- Directory verification using `ls`.

Figure 3 — Verification of the CompanyLab Directory Structure



7. Inspecting Initial Permissions

Before modifying the permissions of the `projects` directory, its initial configuration was inspected.

Command used

`ls -ld /companylab/projects`
The output showed `drwxr-xr-x`, equivalent to numeric permissions `755`.

In this configuration:

- **Owner (`rwx`):** read, write, and execute.
- **Group (`r-x`):** read and execute.
- **Others (`r-x`):** read and execute.

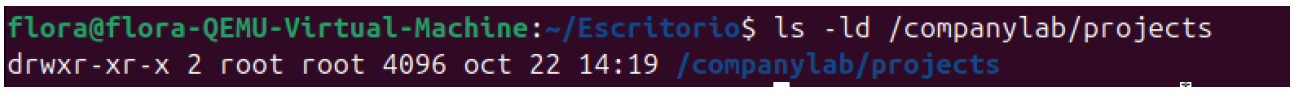
The output also confirmed that both the user and group owner were `root`.

The `-d` option displayed information about the directory itself rather than listing its contents.

Concepts applied

- Permission inspection using `ls -ld`.
- User and group ownership.
- Understanding `rwx` permissions.
- Symbolic and numeric permission notation.

Figure 4 — Inspection of Initial Permissions on the projects Directory



8. Changing Group Ownership

To prepare the shared directory for access by `projectteam` members, its group ownership was changed.

Command used

`sudo chgrp projectteam /companylab/projects`
The `chgrp` command changed the directory's group owner without modifying its user owner or permissions.

The configuration was then verified using:

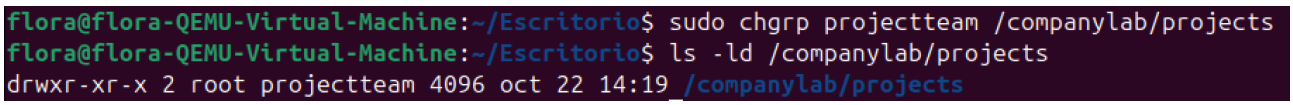
`ls -ld /companylab/projects`
The output confirmed that `root` remained the user owner, while the group owner had changed to `projectteam`.

Permissions remained set to `755`, so further changes were required to restrict access for other users.

Concepts applied

- Changing group ownership using `chgrp`.
- User ownership versus group ownership.
- Ownership verification using `ls -ld`.
- Difference between ownership and permissions.

Figure 5 — Changing the Group Ownership of the projects Directory



9. Configuring Numeric Permissions

After assigning `projectteam` as the group owner, the permissions of the `projects` directory were modified to restrict access for users outside the project team.

Command used

`sudo chmod 770 /companylab/projects`
The `chmod` command modifies file and directory permissions. In this case, numeric notation `770` was used.

This configuration represents:

- **Owner (7 = rwx):** read, write, and execute.
- **Group (7 = rwx):** read, write, and execute.
- **Others (0 = ---):** no permissions.

The configuration was then verified using:

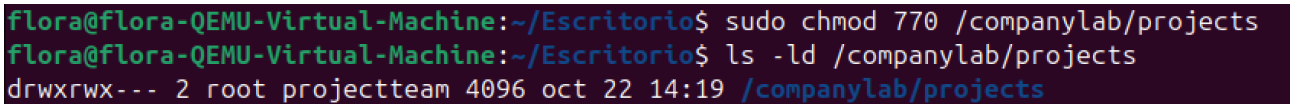
```
ls -ld /companylab/projects
```

The output showed `drwxrwx---`, confirming that `root` and members of `projectteam` had access to the directory, while other users had no permissions assigned.

Concepts applied

- Permission modification using `chmod`.
- Numeric (octal) notation.
- Read (`r`), write (`w`), and execute (`x`) permissions.
- Restricting access for other users.

Figure 6 — Configuring and Verifying 770 Permissions on projects



10. Testing Directory Access

After configuring `770` permissions on the `projects` directory, access tests were performed to verify that only members of `projectteam` could access the resource.

The `sudo -u` command was used to execute commands under different user identities.

Commands used

```
sudo -u ana ls /companylab/projects
sudo -u carlos ls /companylab/projects
sudo -u pedro ls /companylab/projects
```

The following results were observed:

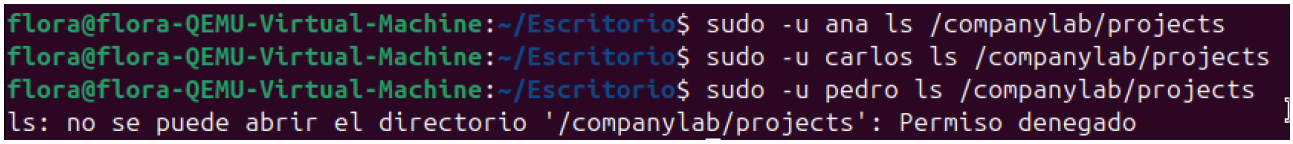
- **Ana:** access granted. No content was displayed because the directory was empty.
- **Carlos:** access granted, with no content to list.
- **Pedro:** access denied with a `Permission denied` message.

These tests confirmed that Ana and Carlos could access the directory through their membership in `projectteam`, while Pedro did not have the required permissions.

Concepts applied

- Effective permission verification.
- Executing commands using `sudo -u`.
- Group-based access control.
- Restricting unauthorized access.

Figure 7 — Directory Access Tests with Ana, Carlos, and Pedro



11. Creating and Verifying a File

To continue practicing permission management, a file was created inside the shared `projects` directory using Ana's identity.

Command used

`sudo -u ana touch /companylab/projects/reporte.txt`
The `touch` command created an empty file named `reporte.txt`, while `sudo -u ana` executed the operation as Ana.

The file was then verified using:

`sudo -u ana ls -l /companylab/projects`
The output confirmed that the file was created with the following configuration:

- **User owner:** `ana`
- **Group owner:** `ana`
- **Permissions:** `rw-rw-r--` (`664`)

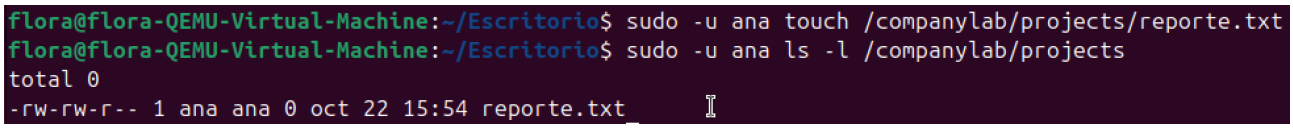
Although the directory belonged to the `projectteam` group, the file was created with Ana's primary group.

Concepts applied

- File creation using `touch`.

- Executing commands as another user.
- File ownership in Linux.
- Difference between directory and file permissions.

Figure 8 — Creating and Verifying reporte.txt



12. Changing File Ownership

To practice file ownership management in Linux, the user owner of `reporte.txt` was changed to Carlos.

Command used

`sudo chown carlos /companylab/projects/reporte.txt`
The `chown` command changes the owner of a file or directory.

The change was then verified using:

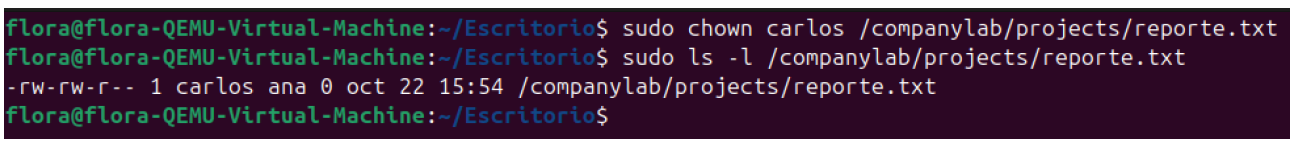
`sudo ls -l /companylab/projects/reporte.txt`
The output confirmed that `carlos` was the new file owner, while the group owner remained `ana`.

Permissions remained set to `rw-rw-r--` (`664`), since changing ownership did not modify the existing permission bits.

Concepts applied

- Changing ownership using `chown`.
- User owner versus group owner.
- Ownership verification using `ls -l`.
- Difference between ownership and permissions.

Figure 9 — Changing and Verifying Ownership of reporte.txt



13. Configuring Numeric and Symbolic Permissions

To practice different methods of permission management in Linux, the permissions of `reporte.txt` were modified using numeric and symbolic notation.

First, permissions were set to `640` using `chmod`.

Command used

`sudo chmod 640 /companylab/projects/reporte.txt`
The configuration was then verified using:

`sudo ls -l /companylab/projects/reporte.txt`
The output showed `rw-r-----`, where:

- **Owner (6 = rw-):** read and write.
- **Group (4 = r--):** read only.
- **Others (0 = ---):** no permissions.

Next, symbolic notation was used to add write permission for the group owner.

`sudo chmod g+w /companylab/projects/reporte.txt`
In this command, `g` represents the group, `+` adds a permission, and `w` represents write access.

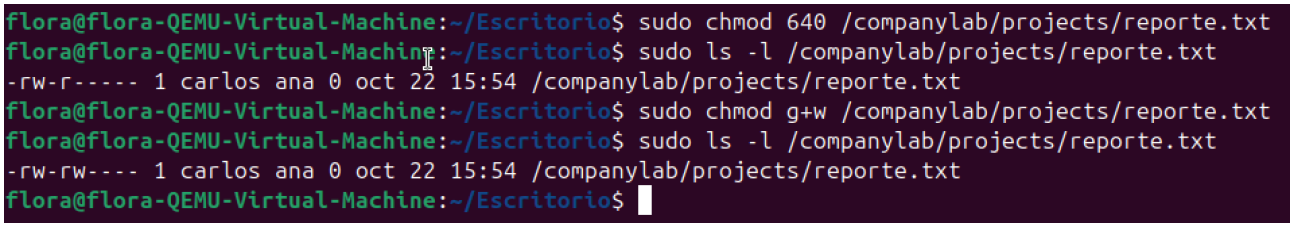
The configuration was verified again using:

`sudo ls -l /companylab/projects/reporte.txt`
The output showed `rw-rw----` (`660`), confirming that write permission had been added for the group owner.

Concepts applied

- Permission configuration using `chmod`.
- Numeric and symbolic notation.
- Modifying specific permissions using `g+w`.
- Permission verification using `ls -l`.

Figure 10 — Configuring and Verifying Numeric and Symbolic Permissions on reporte.txt



14. Tests Performed

The following tasks and configurations were successfully verified during the lab:

- Creation and verification of three local users.
- Creation of the `projectteam` group.
- Addition of Ana and Carlos to the group.
- Verification that Pedro was not a member of `projectteam`.
- Creation of the CompanyLab directory structure.
- Inspection of the initial permissions on the `projects` directory.
- Modification of group ownership using `chgrp`.
- Configuration of `770` permissions using `chmod`.
- Successful directory access for Ana and Carlos.
- Access denial for Pedro with a `Permission denied` message.
- Creation of `reporte.txt` using Ana's identity.
- Modification of file ownership using `chown`.
- Configuration of `640` and `660` permissions using numeric and symbolic notation.

15. Analysis

This lab provided practical experience with how Linux uses users, groups, and permissions to manage access to files and directories.

One of the main concepts explored was group-based permission management. By adding Ana and Carlos to `projectteam` and assigning this group to the `projects` directory, access could be granted without configuring individual permissions for each user.

The access tests demonstrated how `770` permissions work: Ana and Carlos were able to access the directory, while Pedro received a permission-denied message.

The lab also highlighted the difference between directory ownership and file ownership. Although `projects` belonged to the `projectteam` group, `reporte.txt` was created with Ana as its user owner and `ana` as its group owner.

Changing ownership with `chown` then allowed the file to be assigned to Carlos without changing its group ownership.

Finally, `chmod` was used to configure permissions through two methods: numeric notation, which defines a complete permission set, and symbolic notation, which allows specific permissions to be added or removed.

16. Conclusion

This lab provided practical experience with fundamental Linux user, group, and permission management concepts using the Bash terminal.

A workplace scenario was created in which users were organized into a workgroup and access permissions were configured for a shared directory.

The `adduser`, `groupadd`, and `usermod` commands were used to manage accounts and group memberships, while `chgrp`, `chown`, and `chmod` were used to modify resource ownership and permissions.

Access tests confirmed that the configuration allowed authorized users to access the shared directory while restricting access for users outside the assigned group.

These skills provide an important foundation for IT support and Linux system administration, where proper user, group, and permission management is essential for protecting system resources.